Marwadi University	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Subject: Digital Design using Verilog (01CT1619)	**Aim: Write the Verilog code to implement logic functions using continuous assignments.**	
Experiment No: 01	**Date:**	**Enrollment No: 92301733025**

Aim: Write the Verilog code to implement given logic functions using continuous assignments.

g= x1x3 + x2x4

h = (x1 + x3 bar)(x2bar + x4)

f = g + h

IDE: EDA Playground

Theory:

Binary operation performs by any digital circuit with the set of elements 0 and 1 ,are called logical operations or logic functions. The algebra used to symbolically represent the logic function is called Boolean algebra. It is a two state algebra invented by George Boole in 1854.

Thus, a Boolean algebra is a system of mathematics logic for the analysis and designing of digital systems.

A variable or function of variables in Boolean algebra can assume only two values ,either a '0' or a '1'.Hence,there are no fractions, no negative numbers, no square roots, no cube roots, no logarithms etc.

Pre Lab Exercise:

a. What is logic function?
   A logic function is a mathematical expression that describes the relationship between binary inputs (0 or 1) and the resulting binary output (0 or 1) using logical operations such as AND, OR, NOT, NAND, NOR, XOR.

b. What is the minimum number of two-input NAND gates used to perform the function of two input OR gate ?
   Minimum number of 2-input NAND gates required = 3

c. A student makes a mistake somewhere in the process of simplifying the following Boolean expression:

$$AB + A(B + C)$$

$$AB + AB + C$$

$$AB + C$$

Determine where the mistake was made, and what the proper sequence of steps should be to simplify the original expression.

The distributive law was applied incorrectly.
Correct distributive law:

(+) = +

The student wrote AB + C instead of AB + AC, missing the multiplication of A with C.

Step 1: Apply distributive law

+ (+) = + +

Step 2: Apply idempotent law

+ =
+

Step 3: Factor A (optional simplified form)

(+)

Verilog Code:

design.sv

```
// Code your design here
module logics(x1,x2,x3,x4,g,h,f);
input x1, x2, x3, x4;
 output wire g, h, f;

 assign g = (x1 & x3) | (x2 & x4);
  assign h = (x1 | ~x3) & (~x2 | x4);
 assign f = g | h;
endmodule
```

Testbench:

Marwadi University	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Subject: Digital Design using Verilog (01CT1619)	**Aim: Write the Verilog code to implement logic functions using continuous assignments.**	
Experiment No: 01	**Date:**	**Enrollment No: 92301733025**

testbench.sv

```
// Code your testbench here
// or browse Examples
module tb();

 reg x1, x2, x3, x4;
 wire g, h, f;
 logics uut (.x1(x1),.x2(x2),.x3(x3),.x4(x4),.g(g),.h(h),.f(f) );

 initial begin
 $monitor("x1=%b x2=%b x3=%b x4=%b | g=%b h=%b f=%b", x1, x2, x3, x4, g, h, f);
 // Applying different input combinations
 x1 = 0; x2 = 0; x3 = 0; x4 = 0; #10;
 x1 = 0; x2 = 0; x3 = 0; x4 = 1; #10;
 x1 = 0; x2 = 0; x3 = 1; x4 = 0; #10;
 x1 = 0; x2 = 0; x3 = 1; x4 = 1; #10;
 x1 = 0; x2 = 1; x3 = 0; x4 = 0; #10;
 x1 = 0; x2 = 1; x3 = 0; x4 = 1; #10;
 x1 = 0; x2 = 1; x3 = 1; x4 = 0; #10;
 x1 = 0; x2 = 1; x3 = 1; x4 = 1; #10;
 x1 = 1; x2 = 0; x3 = 0; x4 = 0; #10;
 x1 = 1; x2 = 0; x3 = 0; x4 = 1; #10;
 x1 = 1; x2 = 0; x3 = 1; x4 = 0; #10;
 x1 = 1; x2 = 0; x3 = 1; x4 = 1; #10;
 x1 = 1; x2 = 1; x3 = 0; x4 = 0; #10;
 x1 = 1; x2 = 1; x3 = 0; x4 = 1; #10;
 x1 = 1; x2 = 1; x3 = 1; x4 = 0; #10;
 x1 = 1; x2 = 1; x3 = 1; x4 = 1; #10;
 $finish;
 end
endmodule
```

Results:

```
x1=0 x2=0 x3=0 x4=0 | g=0 h=1 f=1
x1=0 x2=0 x3=0 x4=1 | g=0 h=1 f=1
x1=0 x2=0 x3=1 x4=0 | g=0 h=0 f=0
x1=0 x2=0 x3=1 x4=1 | g=0 h=0 f=0
x1=0 x2=1 x3=0 x4=0 | g=0 h=0 f=0
x1=0 x2=1 x3=0 x4=1 | g=1 h=1 f=1
x1=0 x2=1 x3=1 x4=0 | g=0 h=0 f=0
x1=0 x2=1 x3=1 x4=1 | g=1 h=0 f=1
x1=1 x2=0 x3=0 x4=0 | g=0 h=1 f=1
x1=1 x2=0 x3=0 x4=1 | g=0 h=1 f=1
x1=1 x2=0 x3=1 x4=0 | g=1 h=1 f=1
x1=1 x2=0 x3=1 x4=1 | g=1 h=1 f=1
x1=1 x2=1 x3=0 x4=0 | g=0 h=0 f=0
x1=1 x2=1 x3=0 x4=1 | g=1 h=1 f=1
x1=1 x2=1 x3=1 x4=0 | g=1 h=0 f=1
x1=1 x2=1 x3=1 x4=1 | g=1 h=1 f=1
testbench.sv:28: $finish called at 160 (1s)
```

Observation and Result Analysis:

The Verilog module correctly implements a combinational logic circuit using Boolean expressions. Intermediate outputs g and h are formed using AND, OR, and NOT operations, and the final output f is

obtained by OR-ing these results. The design is simple, efficient, and produces outputs that depend only on the present inputs, confirming correct combinational logic behavior.

Post Lab Exercise:

a. An engineer hands you a piece of paper with the following Boolean expression on it, and tells you to build a gate circuit to perform that function:

$$A\overline{B} + \overline{C}(A + B)$$

Write verilog code and test bench for the same. Also Mention Output for the same.

Verilog code:

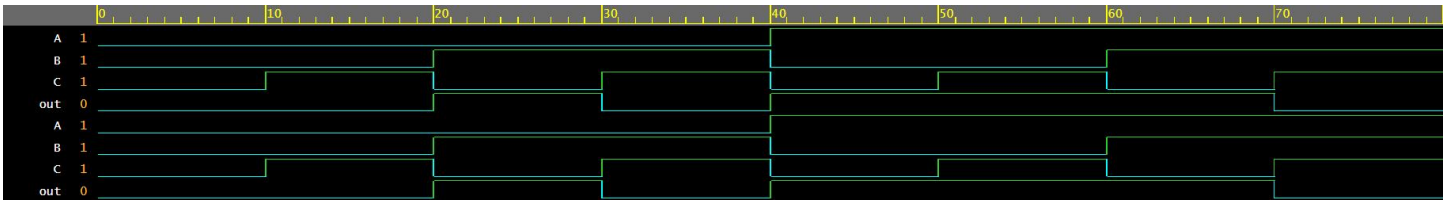
design.sv

```
// Code your design here
module logic(
  input A,B,C;
  output out;
);
  assign out= (A & ~B) | (~C & (A | B));
endmodule
```

Testbench:

```
// Code your testbench here
// or browse Examples
module tb_logic;
  reg A, B, C;
  wire out;
  logic uut(.A(A), .B(B), .C(C), .out(out));

  initial begin
    $dumpfile("logic.vcd");
    $dumpvars(0, logic_tb);

    A=0;B=0;C=0; #10;
    A=0;B=0;C=1; #10;
    A=0;B=1;C=0; #10;
    A=0;B=1;C=1; #10;
    A=1;B=0;C=0; #10;
    A=1;B=0;C=1; #10;
    A=1;B=1;C=0; #10;
    A=1;B=1;C=1; #10;

    $finish;
  end
endmodule
```

Result:



b. The Boolean function (x + y) ($\bar{x}$ + z) (y+ z) is equal to which one of the following expressions?

(a) (x + y)(y+z) (b) ($\bar{x}$ + z) (y+ z) (c) (x + y) ($\bar{x}$ + z) (d) (x + y) ($\bar{x}$ + z)

Write verilog code for given Boolean function and the answer which you have selected from options, Comment on the output for the same.

(c) (x + y) ($\bar{x}$ + z)

Verilog code:

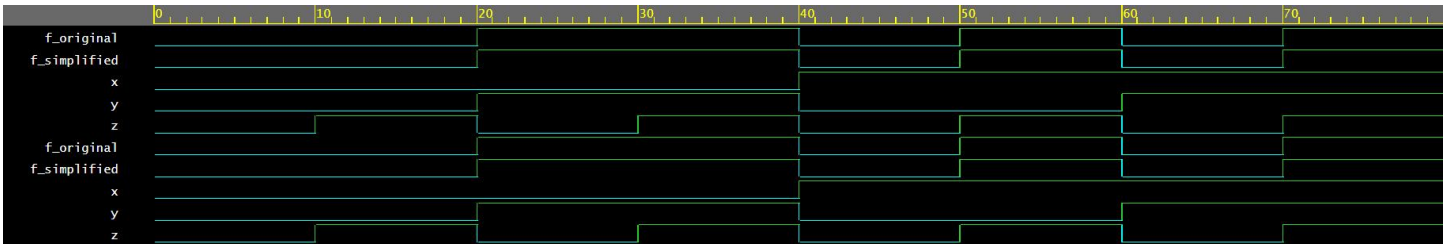
Marwadi University	**Marwadi University** **Faculty of Engineering and Technology** **Department of Information and Communication Technology**	
Subject: Digital Design using Verilog (01CT1619)	**Aim: Write the Verilog code to implement logic functions using continuous assignments.**	
Experiment No: 01	**Date:**	**Enrollment No: 92301733025**

```
// Code your design here
module boolean_func(
    input x, y, z,
    output f_original,
    output f_simplified
);

// original expression
assign f_original = (x | y) & (~x | z) & (y | z);

// simplified expression (option c)
assign f_simplified = (x | y) & (~x | z);

endmodule
```

Testbench:

```
// Code your testbench here
// or browse Examples
module tb_boolean;

reg x, y, z;
wire f_original, f_simplified;

boolean_func uut(
    .x(x),
    .y(y),
    .z(z),
    .f_original(f_original),
    .f_simplified(f_simplified)
);

initial begin
    $dumpfile("boolean.vcd");
    $dumpvars(0, tb_boolean);

    x=0; y=0; z=0; #10;
    x=0; y=0; z=1; #10;
    x=0; y=1; z=0; #10;
    x=0; y=1; z=1; #10;
    x=1; y=0; z=0; #10;
    x=1; y=0; z=1; #10;
    x=1; y=1; z=0; #10;
    x=1; y=1; z=1; #10;

    $finish;
end

endmodule
```

Results:



The outputs of both the original and simplified Boolean expressions are identical for all possible input combinations of x, y, and z. Therefore, the simplified expression $(x + y)(\bar{x} + z)$ correctly represents the given Boolean function, reducing hardware complexity by eliminating one logic term.